

Final Documentation — BE- **02: Connecting to the** **Database**

FlyRank AI

Name: Jiya yadav

Assignment Name: Connecting to the Database

Code: BE-02

Track: Backend AI Engineering

Week: Week 3

Contents

1. Project Overview.....	4
Before vs After.....	4
2. Architecture.....	4
System Diagram.....	4
Layer Responsibilities	5
3. Project Structure.....	5
4. Database Design.....	5
Schema	5
Column Reference.....	6
Seed Data.....	6
5. API Reference	6
Base URL	6
Interactive Docs	6
Endpoints	7
Query Parameters:	7
Example Requests:	8
GET /tasks	8
200 Response:	8
GET /tasks/{id}	8
200 Response:	9
404 Response:	9
POST /tasks.....	9
Request Body:	9
201 Response:	9
400 Response (missing title):.....	10
PUT /tasks/{id}	10
Request Body:	10
200 Response:	10
404 Response:	11
DELETE /tasks/{id}.....	11

200 Response:	11
404 Response:	11
GET /stats	11
200 Response:	11
Status Code Summary	11
6. SQL Queries Reference	12
Queries Explored Manually in DB Browser.....	12
7. Testing Guide	17
Start the Server.....	17
Full Test Suite Using PowerShell	17
8. Results Table	20
9. Key Learnings.....	23
Concepts Learned.....	24
Architecture Insight.....	24
What Transfers to Production	24
10. Requirements Checklist.....	24
Core Requirements.....	24
Optional Extras	25
Stage Checkpoints.....	25
📁 Dependencies.....	25
🔗 Resources Used.....	26

1. Project Overview

What This Project Is

A production-style RESTful Task Management API built with FastAPI and backed by a SQLite database. This project is a direct evolution of the Week 2 in-memory API. The core goal of this assignment was to prove one of the most important ideas in backend engineering:

Before vs After

Week 2 Architecture:

Client —► FastAPI Routes —► Python List —► ✕ Lost on restart

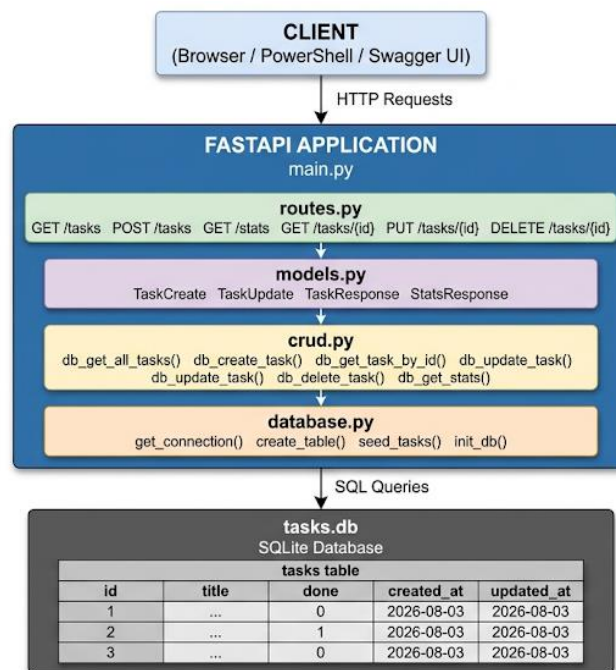
Week 3 Architecture:

Client —► FastAPI Routes —► SQLite Database —► ✓ Persists forever

The client sees zero difference. Every URL, request body, and response format stayed identical. Only the storage layer changed.

2. Architecture

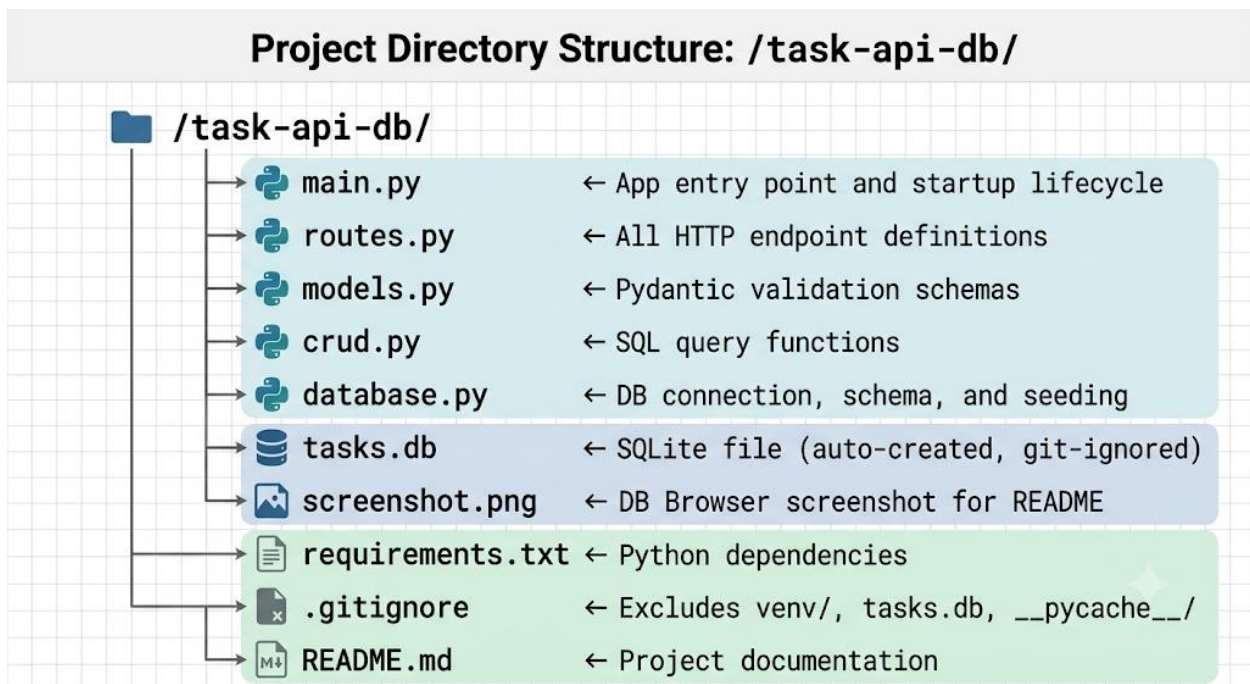
System Diagram



Layer Responsibilities

Layer	File	Responsibility
Entry Point	main.py	Starts the app, runs <code>init_db()</code> on startup
Routes	routes.py	Defines HTTP endpoints, handles status codes
Models	models.py	Validates request bodies, shapes responses
CRUD	crud.py	Contains all SQL queries, talks to the database
Database	database.py	Manages connection, creates table, seeds data
Storage	tasks.db	The actual SQLite database file

3. Project Structure



4. Database Design

Schema

```
CREATE TABLE IF NOT EXISTS tasks (  
    id    INTEGER PRIMARY KEY AUTOINCREMENT,  
    title TEXT NOT NULL,  
    done  INTEGER NOT NULL DEFAULT 0,
```

```

        created_at TEXT DEFAULT (datetime('now')),
        updated_at TEXT DEFAULT (datetime('now'))
    );

```

Column Reference

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for each task
title	TEXT	NOT NULL	Task description, cannot be empty
done	INTEGER	NOT NULL, DEFAULT 0	0 = pending, 1 = completed
created_at	TEXT	DEFAULT datetime('now')	Timestamp when task was created
updated_at	TEXT	DEFAULT datetime('now')	Timestamp when task was last modified

Seed Data

Three tasks are inserted only when the table is empty (first run only):

```

seed_data = [
    ("Learn SQL fundamentals", 0), # pending
    ("Connect SQLite to FastAPI server", 1), # completed
    ("Build an AI feature with Claude", 0), # pending
]

```

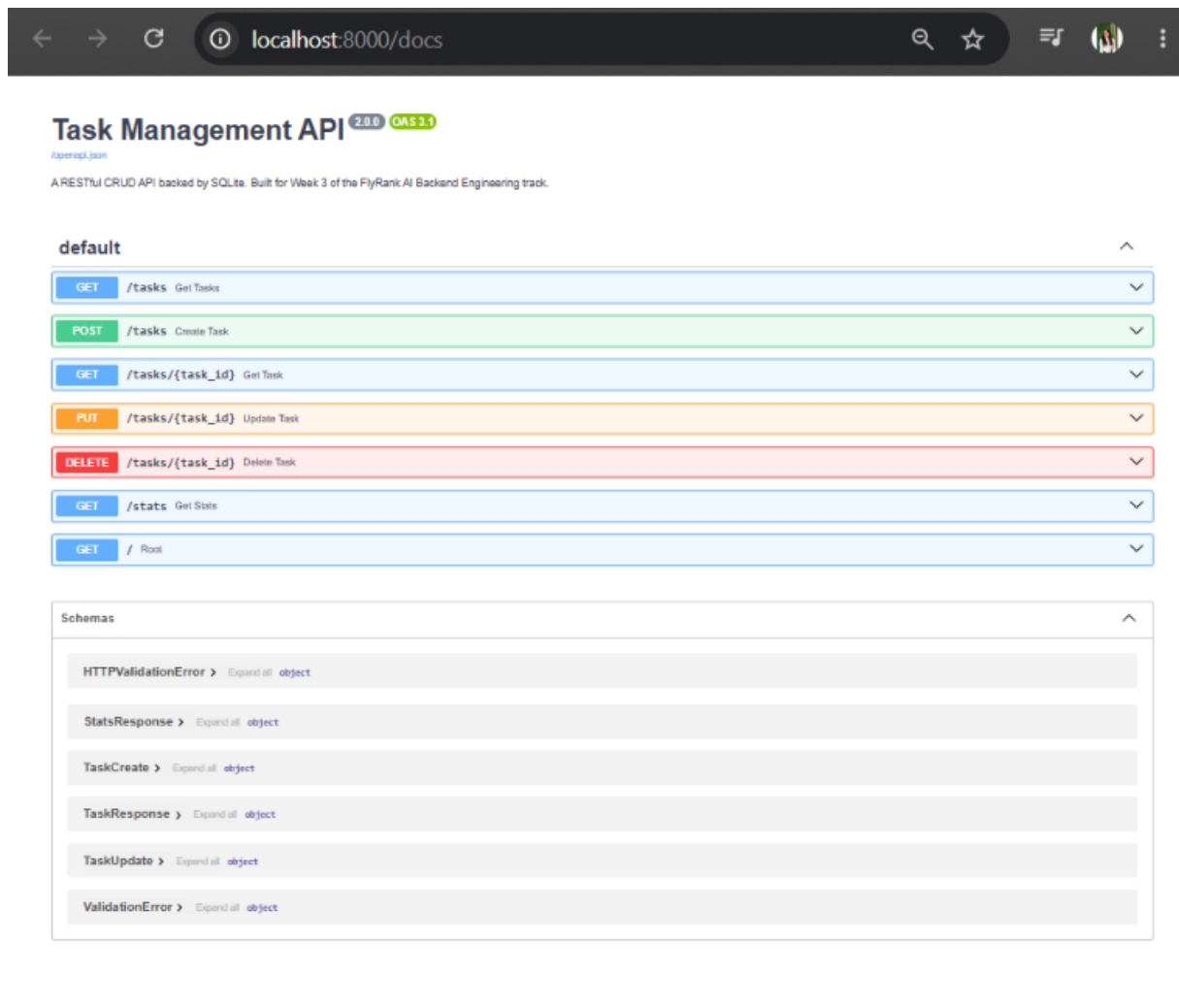
5. API Reference

Base URL

<http://localhost:8000>

Interactive Docs

<http://localhost:8000/docs> ← Swagger UI



Endpoints

GET /tasks

Returns all tasks. Supports optional filtering and sorting.

Query Parameters:

Parameter	Type	Default	Description
search	string	—	Filter tasks whose title contains this keyword (SQL LIKE)
done	boolean	—	Filter by completion status
sort_alpha	boolean	false	Sort results alphabetically by title

Example Requests:

GET /tasks

GET /tasks?done=false

GET /tasks?search=sql

GET /tasks?done=true&sort_alpha=true

GET /tasks

200 Response:

```
[
  {
    "id": 1,
    "title": "Learn SQL fundamentals",
    "done": false,
    "created_at": "2026-08-03 20:16:59",
    "updated_at": "2026-08-03 20:16:59"
  },
  {
    "id": 2,
    "title": "Connect SQLite to FastAPI server",
    "done": true,
    "created_at": "2026-08-03 20:16:59",
    "updated_at": "2026-08-03 20:16:59"
  }
]
```

GET /tasks/{id}

Returns one task by its ID.

Path Parameter:

Parameter	Type	Description
id	integer	The task's unique ID

200 Response:

```
{
  "id": 1,
  "title": "Learn SQL fundamentals",
  "done": false,
  "created_at": "2026-08-03 20:16:59",
  "updated_at": "2026-08-03 20:16:59"
}
```

404 Response:

```
{
  "detail": "Task not found"
}
```

POST /tasks

Creates a new task and saves it to the database.

Request Body:

```
{
  "title": "Write unit tests",
  "done": false
}
```

Field	Type	Required	Description
title	string	✗ No	Task description (cannot be empty)
done	boolean	✗ No	Defaults to false

201 Response:

```
{
  "id": 4,
```

```
"title": "Write unit tests",  
"done": false,  
"created_at": "2026-08-03 20:25:00",  
"updated_at": "2026-08-03 20:25:00"  
}
```

400 Response (missing title):

```
{  
"detail": "Title is required"  
}
```

PUT /tasks/{id}

Updates an existing task. All fields are optional.

Request Body:

```
{  
"title": "Updated task title",  
"done": true  
}
```

Field	Type	Required	Description
title	string	✗ No	New task title
done	boolean	✗ No	New completion status

200 Response:

```
{  
"id": 1,  
"title": "Updated task title",  
"done": true,  
"created_at": "2026-08-03 20:16:59",  
"updated_at": "2026-08-03 20:30:00"  
}
```

404 Response:

```
{  
  "detail": "Task not found"  
}
```

DELETE /tasks/{id}

Permanently removes a task from the database.

200 Response:

```
{  
  "message": "Task 1 deleted successfully"  
}
```

404 Response:

```
{  
  "detail": "Task not found"  
}
```

GET /stats

Returns task count statistics calculated entirely in SQL.

200 Response:

```
{  
  "total_tasks": 5,  
  "completed_tasks": 2,  
  "pending_tasks": 3  
}
```

Status Code Summary

Code	Meaning	When It Happens
200	OK	Successful GET, PUT, DELETE
201	Created	Successful POST

400	Bad Request	Missing or empty title
404	Not Found	Task ID does not exist

6. SQL Queries Reference

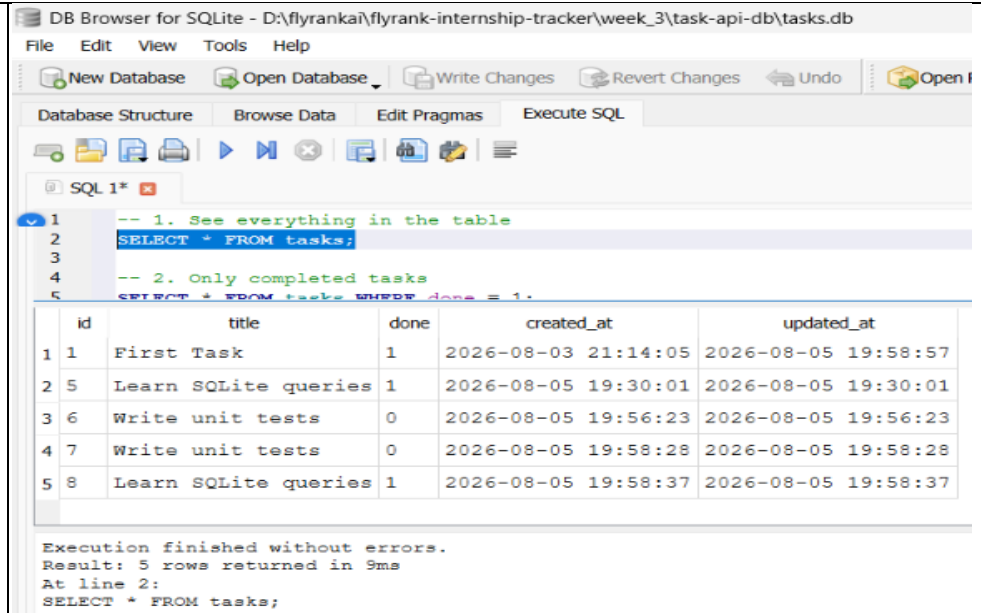
Queries Used Inside the Application

Operation	SQL	Operation
Create table	CREATE TABLE IF NOT EXISTS tasks (...)	Create table
Count rows for seeding	SELECT COUNT(*) AS total FROM tasks	Count rows for seeding
Seed data	INSERT INTO tasks (title, done) VALUES (?, ?)	Seed data
Get all tasks	SELECT id, title, done, created_at, updated_at FROM tasks ORDER BY id ASC	Get all tasks
Get one task	SELECT ... FROM tasks WHERE id = ?	Get one task
Search by title	SELECT ... FROM tasks WHERE title LIKE ?	Search by title
Filter by done	SELECT ... FROM tasks WHERE done = ?	Filter by done
Sort alphabetically	SELECT ... FROM tasks ORDER BY title ASC	Sort alphabetically
Create task	INSERT INTO tasks (title, done) VALUES (?, ?)	Create task
Update task	UPDATE tasks SET title = ?, done = ?, updated_at = datetime('now') WHERE id = ?	Update task
Delete task	DELETE FROM tasks WHERE id = ?	Delete task
Count stats	SELECT COUNT(*), SUM(CASE WHEN done = 1 ...) FROM tasks	Count stats

Queries Explored Manually in DB Browser

-- 1. See everything in the table

SELECT *
FROM tasks;



The screenshot shows the DB Browser for SQLite interface. The SQL editor contains the following code:

```
1 -- 1. See everything in the table
2 SELECT * FROM tasks;
3
4 -- 2. Only completed tasks
5 SELECT * FROM tasks WHERE done = 1;
```

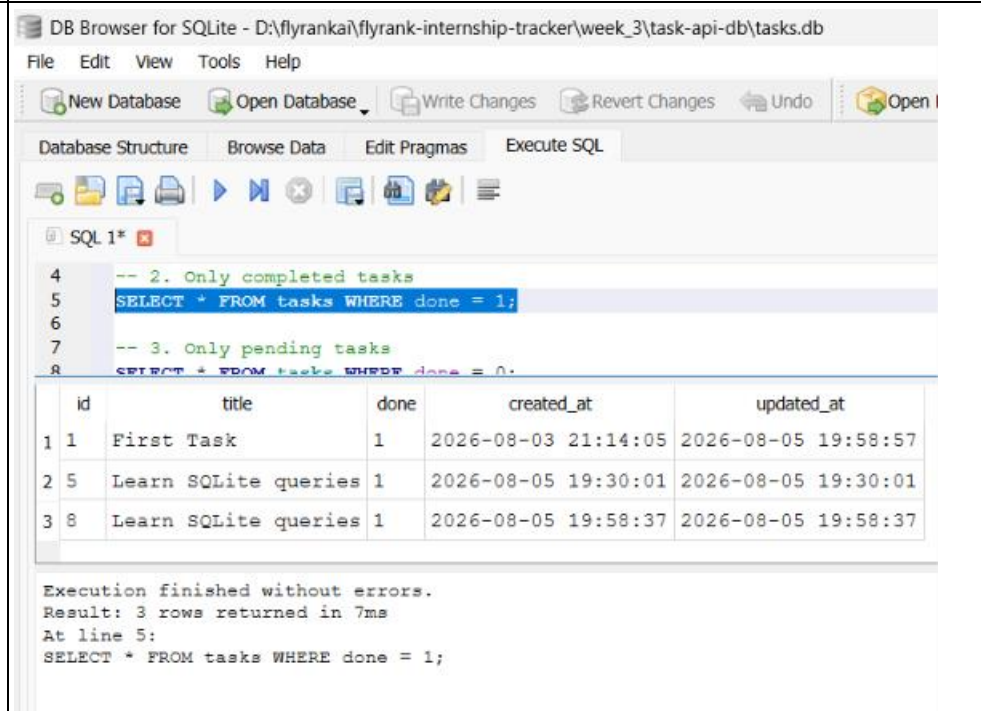
The results pane displays a table with 5 rows and 6 columns:

	id	title	done	created_at	updated_at
1	1	First Task	1	2026-08-03 21:14:05	2026-08-05 19:58:57
2	5	Learn SQLite queries	1	2026-08-05 19:30:01	2026-08-05 19:30:01
3	6	Write unit tests	0	2026-08-05 19:56:23	2026-08-05 19:56:23
4	7	Write unit tests	0	2026-08-05 19:58:28	2026-08-05 19:58:28
5	8	Learn SQLite queries	1	2026-08-05 19:58:37	2026-08-05 19:58:37

Execution finished without errors.
Result: 5 rows returned in 9ms
At line 2:
SELECT * FROM tasks;

-- 2. Only completed tasks

SELECT * FROM
tasks WHERE
done = 1;



The screenshot shows the DB Browser for SQLite interface. The SQL editor contains the following code:

```
4 -- 2. Only completed tasks
5 SELECT * FROM tasks WHERE done = 1;
6
7 -- 3. Only pending tasks
8 SELECT * FROM tasks WHERE done = 0;
```

The results pane displays a table with 3 rows and 6 columns:

	id	title	done	created_at	updated_at
1	1	First Task	1	2026-08-03 21:14:05	2026-08-05 19:58:57
2	5	Learn SQLite queries	1	2026-08-05 19:30:01	2026-08-05 19:30:01
3	8	Learn SQLite queries	1	2026-08-05 19:58:37	2026-08-05 19:58:37

Execution finished without errors.
Result: 3 rows returned in 7ms
At line 5:
SELECT * FROM tasks WHERE done = 1;

-- 3. Only pending tasks

```
SELECT *  
FROM tasks  
WHERE done = 0;
```

```
SQL 1*  
1 -- 1. See everything in the table  
2 SELECT * FROM tasks;  
3  
4 -- 2. Only completed tasks  
5 SELECT * FROM tasks WHERE done = 1;  
6  
7 -- 3. Only pending tasks  
8 SELECT * FROM tasks WHERE done = 0;  
9  
10  
11 -- 4. Count total tasks  
12 SELECT COUNT(*) AS total FROM tasks;  
13  
14 -- 5. Search for keyword in title
```

	id	title	done	created_at	updated_at
1	6	Write unit tests	0	2026-08-05 19:56:23	2026-08-05 19:56:23
2	7	Write unit tests	0	2026-08-05 19:58:28	2026-08-05 19:58:28

Execution finished without errors.
Result: 2 rows returned in 7ms
At line 8:
SELECT * FROM tasks WHERE done = 0;

-- 4. Count total tasks

```
SELECT COUNT(*)  
AS total FROM  
tasks;
```

DB Browser for SQLite - D:\flyrankai\flyrank-internship-tracker\week_3\task

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes

Database Structure Browse Data Edit Pragma Execute SQL

```
SQL 1*  
10 -- 4. Count total tasks  
11 SELECT COUNT(*) AS total FROM tasks;  
12  
13 -- 5. Search for keyword in title  
14 SELECT * FROM tasks WHERE title LIKE '%sql%';
```

	total
1	5

Execution finished without errors.
Result: 1 rows returned in 7ms
At line 11:
SELECT COUNT(*) AS total FROM tasks;

-- 5. Search for keyword in title

```
SELECT * FROM  
tasks WHERE title  
LIKE '%sql%';
```

DB Browser for SQLite - D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db\tasks.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Pr

Database Structure Browse Data Edit Pragma Execute SQL

```
SQL 1*  
10 -- 4. Count total tasks  
11 SELECT COUNT(*) AS total FROM tasks;  
12  
13 -- 5. Search for keyword in title  
14 SELECT * FROM tasks WHERE title LIKE '%sql%';
```

	id	title	done	created_at	updated_at
1	5	Learn SQLite queries	1	2026-08-05 19:30:01	2026-08-05 19:30:01
2	8	Learn SQLite queries	1	2026-08-05 19:58:37	2026-08-05 19:58:37

Execution finished without errors.
Result: 2 rows returned in 10ms
At line 14:
SELECT * FROM tasks WHERE title LIKE '%sql%';

-- 6. Sort
alphabetically

SELECT * FROM
tasks ORDER BY
title ASC;

DB Browser for SQLite - D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db\tasks.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project

Database Structure Browse Data Edit Pragmas Execute SQL

SQL 1*

```
13 -- 5. Search for keyword in title
14 SELECT * FROM tasks WHERE title LIKE '%sql%';
15
16 -- 6. Sort alphabetically
17 SELECT * FROM tasks ORDER BY title ASC;
```

	id	title	done	created_at	updated_at
1	1	First Task	1	2026-08-03 21:14:05	2026-08-05 19:58:57
2	5	Learn SQLite queries	1	2026-08-05 19:30:01	2026-08-05 19:30:01
3	8	Learn SQLite queries	1	2026-08-05 19:58:37	2026-08-05 19:58:37
4	6	Write unit tests	0	2026-08-05 19:56:23	2026-08-05 19:56:23
5	7	Write unit tests	0	2026-08-05 19:58:28	2026-08-05 19:58:28

Execution finished without errors.
Result: 5 rows returned in 10ms
At line 17:
SELECT * FROM tasks ORDER BY title ASC;

-- 7. Mark all
tasks
complete

UPDATE tasks SET
done = 1;

DB Browser for SQLite - D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db\tasks.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo

Database Structure Browse Data Edit Pragmas Execute SQL

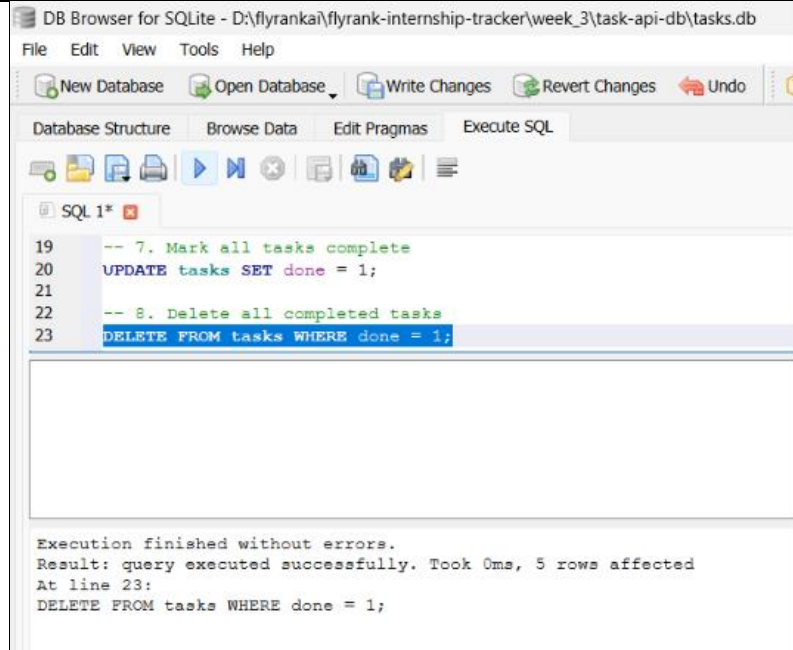
SQL 1*

```
16 -- 6. Sort alphabetically
17 SELECT * FROM tasks ORDER BY title ASC;
18
19 -- 7. Mark all tasks complete
20 UPDATE tasks SET done = 1;
```

Execution finished without errors.
Result: query executed successfully. Took 2ms, 5 rows affected
At line 20:
UPDATE tasks SET done = 1;

-- 8. Delete
all completed
tasks

DELETE FROM
tasks WHERE
done = 1;



The screenshot shows the DB Browser for SQLite interface. The title bar indicates the database path: D:\flyranka\flyrank-internship-tracker\week_3\task-api-db\tasks.db. The menu bar includes File, Edit, View, Tools, and Help. The toolbar has buttons for New Database, Open Database, Write Changes, Revert Changes, and Undo. The main window has tabs for Database Structure, Browse Data, Edit Pragmas, and Execute SQL. The SQL editor shows the following code:

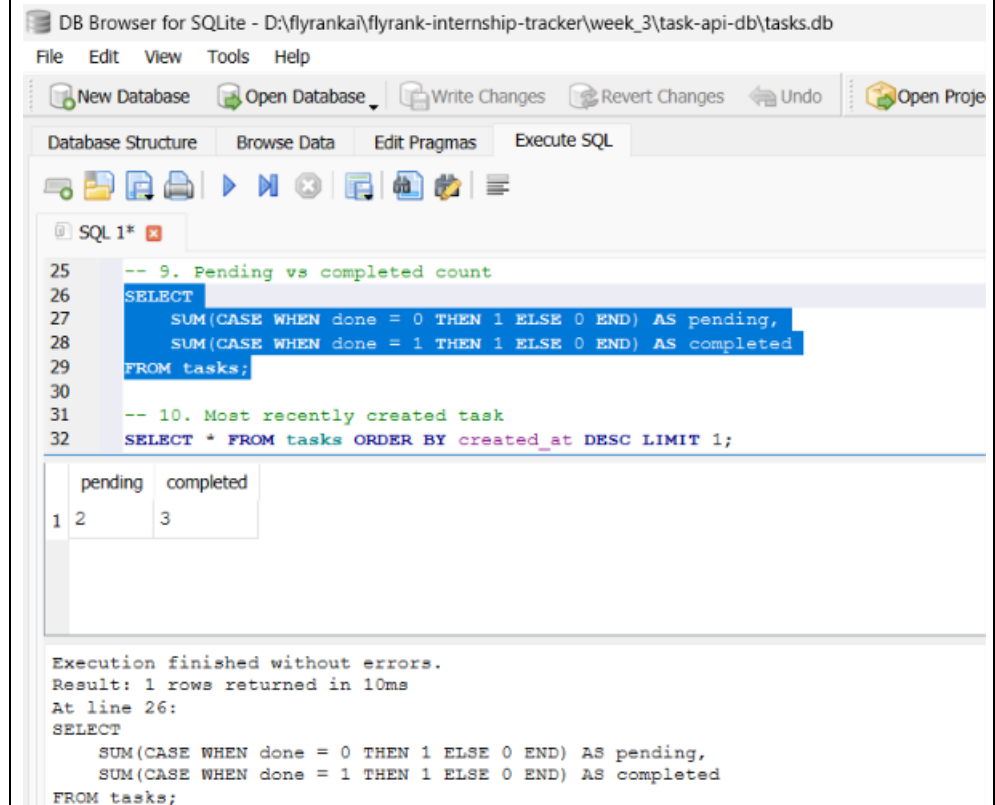
```
19 -- 7. Mark all tasks complete
20 UPDATE tasks SET done = 1;
21
22 -- 8. Delete all completed tasks
23 DELETE FROM tasks WHERE done = 1;
```

Below the editor, the execution status is displayed:

```
Execution finished without errors.
Result: query executed successfully. Took 0ms, 5 rows affected
At line 23:
DELETE FROM tasks WHERE done = 1;
```

-- 9. Pending
vs completed
count

SELECT
SUM(CASE
WHEN done = 0
THEN 1 ELSE 0
END) AS
pending,
SUM(CASE
WHEN done = 1
THEN 1 ELSE 0
END) AS
completed
FROM tasks;



The screenshot shows the DB Browser for SQLite interface. The title bar indicates the database path: D:\flyranka\flyrank-internship-tracker\week_3\task-api-db\tasks.db. The menu bar includes File, Edit, View, Tools, and Help. The toolbar has buttons for New Database, Open Database, Write Changes, Revert Changes, Undo, and Open Project. The main window has tabs for Database Structure, Browse Data, Edit Pragmas, and Execute SQL. The SQL editor shows the following code:

```
25 -- 9. Pending vs completed count
26 SELECT
27     SUM(CASE WHEN done = 0 THEN 1 ELSE 0 END) AS pending,
28     SUM(CASE WHEN done = 1 THEN 1 ELSE 0 END) AS completed
29 FROM tasks;
30
31 -- 10. Most recently created task
32 SELECT * FROM tasks ORDER BY created_at DESC LIMIT 1;
```

Below the editor, the execution status is displayed:

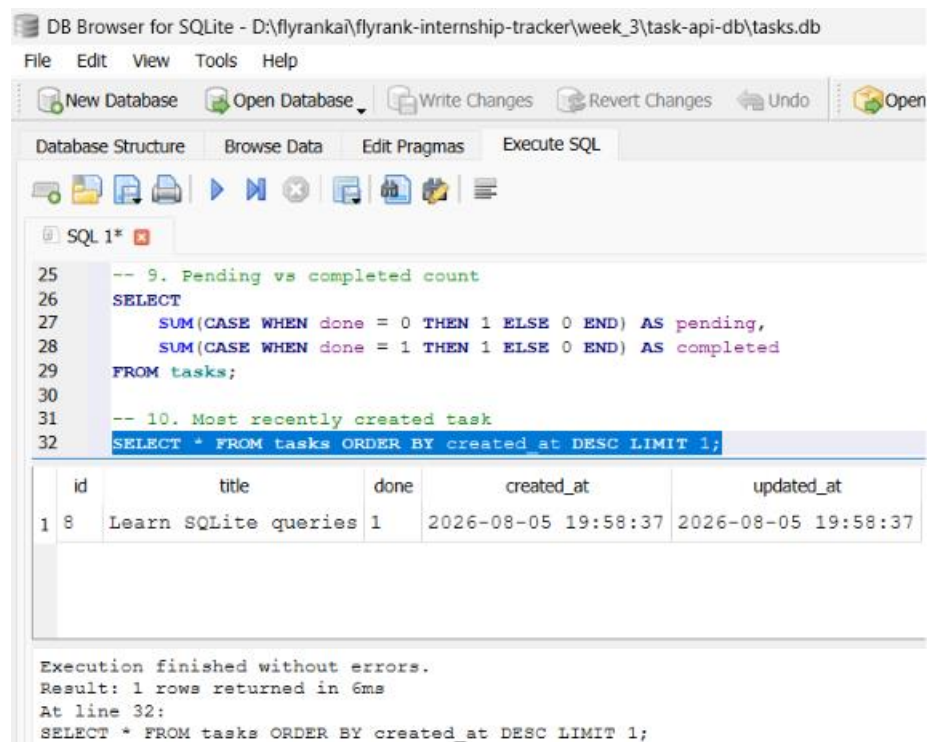
```
Execution finished without errors.
Result: 1 rows returned in 10ms
At line 26:
SELECT
    SUM(CASE WHEN done = 0 THEN 1 ELSE 0 END) AS pending,
    SUM(CASE WHEN done = 1 THEN 1 ELSE 0 END) AS completed
FROM tasks;
```

The result set is displayed in a table:

	pending	completed
1	2	3

-- 10. Most recently created task

```
SELECT * FROM tasks ORDER BY created_at DESC LIMIT 1;
```



The screenshot shows the DB Browser for SQLite application. The title bar indicates the database file is 'D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db\tasks.db'. The 'Execute SQL' tab is active, displaying a SQL query. The query consists of two parts: a comment '-- 9. Pending vs completed count' followed by a SELECT statement with two columns, 'pending' and 'completed', and a second comment '-- 10. Most recently created task' followed by a SELECT statement that is highlighted in blue. Below the query, a table of results is shown with one row. The table has five columns: 'id', 'title', 'done', 'created_at', and 'updated_at'. The row contains the values '1', '8', 'Learn SQLite queries', '1', and '2026-08-05 19:58:37', '2026-08-05 19:58:37'. At the bottom, a status message says 'Execution finished without errors. Result: 1 rows returned in 6ms At line 32: SELECT * FROM tasks ORDER BY created_at DESC LIMIT 1;'.

```
DB Browser for SQLite - D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db\tasks.db
File Edit View Tools Help
New Database Open Database Write Changes Revert Changes Undo Open
Database Structure Browse Data Edit Pragmas Execute SQL
SQL 1*
25 -- 9. Pending vs completed count
26 SELECT
27     SUM(CASE WHEN done = 0 THEN 1 ELSE 0 END) AS pending,
28     SUM(CASE WHEN done = 1 THEN 1 ELSE 0 END) AS completed
29 FROM tasks;
30
31 -- 10. Most recently created task
32 SELECT * FROM tasks ORDER BY created_at DESC LIMIT 1;
id title done created_at updated_at
1 8 Learn SQLite queries 1 2026-08-05 19:58:37 2026-08-05 19:58:37
Execution finished without errors.
Result: 1 rows returned in 6ms
At line 32:
SELECT * FROM tasks ORDER BY created_at DESC LIMIT 1;
```

7. Testing Guide

Start the Server

```
uvicorn main:app --reload --port 8000
```

Full Test Suite Using PowerShell

— READ —

Get all tasks

```
Invoke-RestMethod -Uri "http://localhost:8000/tasks" -Method GET
```

Get one task

```
Invoke-RestMethod -Uri "http://localhost:8000/tasks/1" -Method GET
```

Test 404

```
Invoke-RestMethod -Uri "http://localhost:8000/tasks/999" -Method GET
```

— CREATE —

Create a task

```
Invoke-RestMethod -Uri "http://localhost:8000/tasks" `
-Method POST `
-ContentType "application/json" `
-Body '{"title": "Write unit tests", "done": false}'
```

Create a completed task

```
Invoke-RestMethod -Uri "http://localhost:8000/tasks" `
-Method POST `
-ContentType "application/json" `
-Body '{"title": "Learn SQLite queries", "done": true}'
```

Test 400 — empty title

```
Invoke-RestMethod -Uri "http://localhost:8000/tasks" `
-Method POST `
-ContentType "application/json" `
-Body '{"title": ""}'
```

— UPDATE —

Mark task 1 as done

```
Invoke-RestMethod -Uri "http://localhost:8000/tasks/1" `
-Method PUT `
-ContentType "application/json" `
-Body '{"done": true}'
```

Update title only

```
Invoke-RestMethod -Uri "http://localhost:8000/tasks/2" `
```

-Method PUT `

-ContentType "application/json" `

-Body '{"title": "Updated title"}'

— DELETE —————

Delete task 4

Invoke-RestMethod -Uri "http://localhost:8000/tasks/4" -Method DELETE

Test 404 delete

Invoke-RestMethod -Uri "http://localhost:8000/tasks/3" -Method DELETE

— FILTERS & EXTRAS —————

Search

Invoke-RestMethod -Uri "http://localhost:8000/tasks?search=sql" -Method GET

Filter pending only

Invoke-RestMethod -Uri "http://localhost:8000/tasks?done=false" -Method GET

Filter completed only

Invoke-RestMethod -Uri "http://localhost:8000/tasks?done=true" -Method GET

Sort alphabetically

Invoke-RestMethod -Uri "http://localhost:8000/tasks?sort_alpha=true" -Method GET

Stats

Invoke-RestMethod -Uri "http://localhost:8000/stats" -Method GET

— PERSISTENCE TEST —

Stop server → CTRL+C

Restart server → uvicorn main:app --reload --port 8000

Run this — data must still exist

Invoke-RestMethod -Uri "http://localhost:8000/tasks" -Method GET

8. Results Table

Test	Status	Result
GET /tasks	200	Array of task objects <pre>(venv) PS D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db> Invoke-RestMethod -Uri "http://localhost:8000/tasks" -Method GET</pre> id : 1 title : First Task done : True created_at : 2026-08-03 21:14:05 updated_at : 2026-08-05 19:31:49 id : 4 title : Write unit tests done : False created_at : 2026-08-05 19:29:52 updated_at : 2026-08-05 19:29:52 id : 5 title : Learn SQLite queries done : True created_at : 2026-08-05 19:30:01 updated_at : 2026-08-05 19:30:01 id : 6 title : Write unit tests done : False created_at : 2026-08-05 19:56:23 updated_at : 2026-08-05 19:56:23
GET /tasks/1	200	Single task object <pre>(venv) PS D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db> Invoke-RestMethod -Uri "http://localhost:8000/tasks/1" -Method GET</pre> id : 1 title : First Task done : True created_at : 2026-08-03 21:14:05 updated_at : 2026-08-05 19:31:49

GET /tasks/99	404	<pre>{ "detail": "Task not found" }</pre> <pre>(venv) PS D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db> Invoke- -RestMethod -Uri "http://localhost:8000/tasks/999" -Method GET Invoke-RestMethod : { "detail": "Task not found" } At line:1 char:1 + Invoke-RestMethod -Uri "http://localhost:8000/tasks/999" -Method GET + ~~~~~ + CategoryInfo : InvalidOperation: (System.Net.HttpWebRequest :HttpWebRequest) [Invoke-RestMethod], WebException + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.Powe rShell.Commands.InvokeRestMethodCommand</pre>
POST with title	201	<pre>New task with auto id</pre> <pre>(venv) PS D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db> Invoke- -RestMethod -Uri "http://localhost:8000/tasks" ` >> -Method POST ` >> -ContentType "application/json" ` >> -Body '{"title": "Write unit tests", "done": false}' id : 7 title : Write unit tests done : False created_at : 2026-08-05 19:58:28 updated_at : 2026-08-05 19:58:28</pre>
POST empty title	400	<pre>Validation error</pre> <pre>(venv) PS D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db> Invoke- -RestMethod -Uri "http://localhost:8000/tasks" ` >> -Method POST ` >> -ContentType "application/json" ` >> -Body '{"title": ""}' Invoke-RestMethod : { "detail": "Title is required" } At line:1 char:1 + Invoke-RestMethod -Uri "http://localhost:8000/tasks" ` + ~~~~~ + CategoryInfo : InvalidOperation: (System.Net.HttpWebRequest :HttpWebRequest) [Invoke-RestMethod], WebException + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.Powe rShell.Commands.InvokeRestMethodCommand</pre>

PUT /tasks/1	200	Updated task object <pre>(venv) PS D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db> Invoke -RestMethod -Uri "http://localhost:8000/tasks/1" ` >> -Method PUT ` >> -ContentType "application/json" ` >> -Body '{"done": true}' id : 1 title : First Task done : True created_at : 2026-08-03 21:14:05 updated_at : 2026-08-05 19:58:57</pre>
PUT /tasks/2	404	{"detail": "Task not found"} <pre>(venv) PS D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db> Invoke -RestMethod -Uri "http://localhost:8000/tasks/2" ` >> -Method PUT ` >> -ContentType "application/json" ` >> -Body '{"title": "Updated title"}' Invoke-RestMethod : {"detail":"Task not found"} At line:1 char:1 + Invoke-RestMethod -Uri "http://localhost:8000/tasks/2" ` + ~~~~~ + CategoryInfo : InvalidOperation: (System.Net.HttpWebRequest :HttpWebRequest) [Invoke-RestMethod], WebException + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.Powe rShell.Commands.InvokeRestMethodCommand</pre>
DELETE /tasks/4	200	Success message <pre>(venv) PS D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db> Invoke -RestMethod -Uri "http://localhost:8000/tasks/4" -Method DELETE message ----- Task 4 deleted successfully</pre>
DELETE /tasks/3	404	{"detail": "Task not found"} <pre>(venv) PS D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db> Invoke -RestMethod -Uri "http://localhost:8000/tasks/3" -Method DELETE Invoke-RestMethod : {"detail":"Task not found"} At line:1 char:1 + Invoke-RestMethod -Uri "http://localhost:8000/tasks/3" -Method DELETE + ~~~~~ + CategoryInfo : InvalidOperation: (System.Net.HttpWebRequest :HttpWebRequest) [Invoke-RestMethod], WebException + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.Powe rShell.Commands.InvokeRestMethodCommand</pre>
GET /stats	200	Count object

		<pre>(venv) PS D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db> Invoke -RestMethod -Uri "http://localhost:8000/stats" -Method GET total_tasks completed_tasks pending_tasks ----- 5 3 2</pre>
Restart server	—	All tasks still present

```
PS D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db> venv\Scripts\activate
(venv) PS D:\flyrankai\flyrank-internship-tracker\week_3\task-api-db> uvicorn main:app --reload --port 8000
INFO: Will watch for changes in these directories: ['D:\\flyrankai\\flyrank-internship-tracker\\week_3\\task-api-db']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [16480] using WatchFiles
INFO: Started server process [20592]
INFO: Waiting for application startup.
🔥 Starting Task API...
Table 'tasks' is ready.
Database already has 4 task(s). Skipping seed.
INFO: Application startup complete.
INFO: 127.0.0.1:57058 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:58369 - "GET /tasks HTTP/1.1" 200 OK
INFO: 127.0.0.1:62075 - "GET /tasks/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:62076 - "GET /tasks/999 HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:62077 - "POST /tasks HTTP/1.1" 201 Created
INFO: 127.0.0.1:62078 - "POST /tasks HTTP/1.1" 201 Created
INFO: 127.0.0.1:61024 - "POST /tasks HTTP/1.1" 400 Bad Request
INFO: 127.0.0.1:61025 - "PUT /tasks/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:61026 - "PUT /tasks/2 HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:61027 - "DELETE /tasks/3 HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:58985 - "DELETE /tasks/4 HTTP/1.1" 200 OK
INFO: 127.0.0.1:57869 - "GET /tasks?search=sql HTTP/1.1" 200 OK
INFO: 127.0.0.1:57869 - "GET /tasks?done=false HTTP/1.1" 200 OK
INFO: 127.0.0.1:57871 - "GET /tasks?done=true HTTP/1.1" 200 OK
INFO: 127.0.0.1:57873 - "GET /tasks?sort_alpha=true HTTP/1.1" 200 OK
INFO: 127.0.0.1:57874 - "GET /stats HTTP/1.1" 200 OK
```

9. Key Learnings

The Most Important Idea

- APIs describe WHAT your application does.

- Databases describe WHERE your application stores its data.

These two concerns are completely separate. Changing one does not require changing the other.

Concepts Learned

Concept	What It Means	Where used
Database	Permanent storage that survives restarts	tasks.db
SQLite	Lightweight file-based database, no server needed	database.py
Table	Rows and columns, like a spreadsheet	tasks table
Row	One record in a table	One task
Column	One property of every row	id, title, done
SQL	Language for querying databases	All CRUD operations
Primary Key	Unique identifier for each row	id column
Persistence	Data survives after the program stops	Proven by restart test
Schema	Structure of the database	CREATE TABLE statement
Migration	Controlled schema changes	Will be explored in later weeks

Architecture Insight

- Week 2 Problem: Python list = wiped clean every time the server restarts
- Week 3 Solution: SQLite file = data written to disk, survives forever
- The API layer stayed 100% identical.
- This separation is the foundation of backend engineering.

What Transfers to Production

Skill Learned Here	Transfers To
SQL SELECT queries	PostgreSQL, MySQL, any relational database
SQL INSERT / UPDATE / DELETE	Same syntax works everywhere
Separating routes from queries	Standard pattern in all frameworks
Pydantic validation	Used in production FastAPI apps
IF NOT EXISTS table creation	Database migration patterns

10. Requirements Checklist

Core Requirements

- ✓ API exposes same CRUD endpoints as Assignment 1
- ✓ Tasks stored in SQLite instead of memory

- ✓ Data survives server restarts
- ✓ Database file auto-created if missing
- ✓ Tasks table auto-created if missing
- ✓ Three example tasks inserted only on first run
- ✓ All CRUD operations use SQL queries
- ✓ Unknown ids return 404
- ✓ Invalid requests return 400
- ✓ Public GitHub repository with README and screenshot

Optional Extras

- ✓ GET /tasks?search= using SQL LIKE operator
- ✓ GET /tasks?done= using SQL WHERE clause
- ✓ GET /tasks?sort_alpha= sorted by title
- ✓ GET /stats using SQL COUNT()
- ✓ created_at and updated_at timestamps on every task

Stage Checkpoints

- ✓ Stage 0 — Server restart shows seed tasks only once
- ✓ Stage 1 — GET /tasks returns database contents
- ✓ Stage 2 — Tasks survive server restart after POST
- ✓ Stage 3 — Create, update, delete all confirmed via GET
- ✓ Stage 4 — Manual SQL changes reflected immediately in API
- ✓ Stage 5 — README complete with screenshot and SQL example

Dependencies

fastapi==0.111.0

uvicorn==0.29.0

pydantic==2.7.1

sqlite3 — built into Python, no installation needed

Resources Used

Resource	Purpose
FastAPI Docs	Framework reference
SQLite Docs	SQL syntax reference
DB Browser for SQLite	Visual database inspection
Pydantic Docs	Validation schema reference